

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 2

## Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

|                     |                                               |             |
|---------------------|-----------------------------------------------|-------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY: 2026–27 |

## 1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

## 2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

## 3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

## 4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size:  $28 \times 28$

## 5. Image Preprocessing

Fashion-MNIST images are stored as  $28 \times 28$  matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to  $[0,1]$ .
5. Convert labels into one-hot vectors.

## 6. Experimental Procedure

### Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

**Expected Output:** Dataset summary and sample images.

### Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

### Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

### Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

### Task 5: Model Evaluation

Compute

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

## Source code:

GitHub Repository: [https://github.com/un-shreeya/DeepLearning\\_Lab](https://github.com/un-shreeya/DeepLearning_Lab)

## 7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

| Hyperparameter          | Candidate Values    |
|-------------------------|---------------------|
| Hidden Layers           | 1, 2, 3             |
| Hidden Neurons          | 32, 64, 128, 256    |
| Learning Rate           | 0.1, 0.01, 0.001    |
| Batch Size              | 16, 32, 64, 128     |
| Epochs                  | 10, 20, 30          |
| Optimizer               | SGD, Adam, RMSProp  |
| Activation Function     | ReLU, Tanh, Sigmoid |
| Dropout Rate (Optional) | 0.0, 0.2, 0.5       |

## Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

## 8. Mandatory Plots

Generate the following plots.

1. Sample Images
2. Class Distribution
3. Training Accuracy vs Epoch
4. Validation Accuracy vs Epoch
5. Training Loss vs Epoch
6. Validation Loss vs Epoch
7. Confusion Matrix

8. Hyperparameter Search Results
  9. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

## Generated Plots and Interpretation

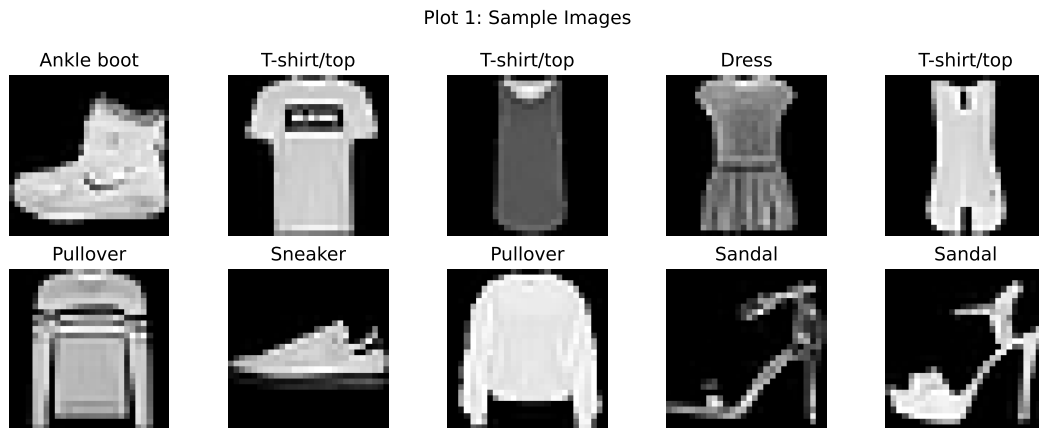


Figure 1: Sample Images from Fashion-MNIST Dataset

**Inference:** The plot shows a random sample of 10 images from the Fashion-MNIST dataset. The grayscale images are successfully loaded and mapped to their corresponding labels.

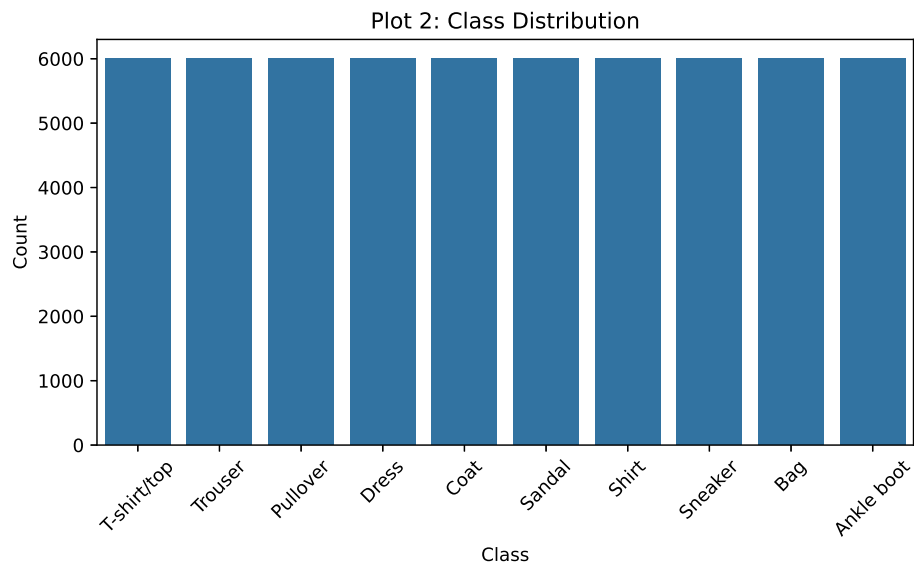


Figure 2: Class Distribution of Training Data

**Inference:** The training dataset exhibits a uniform class distribution across all 10 categories, with each class containing 6,000 samples.

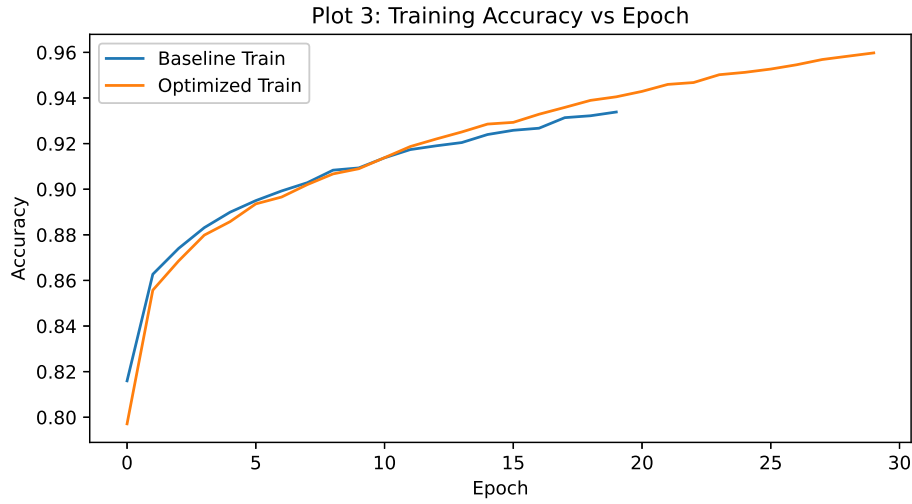


Figure 3: Training Accuracy vs Epoch

**Inference:** The baseline model reaches a training accuracy around 94%. The optimised model reaches an accuracy of around 96% by 30 epochs. Since the optimised model has higher capacity (2 hidden layers, 256 neurons) and trains for longer, it achieves a higher training accuracy than the baseline model.

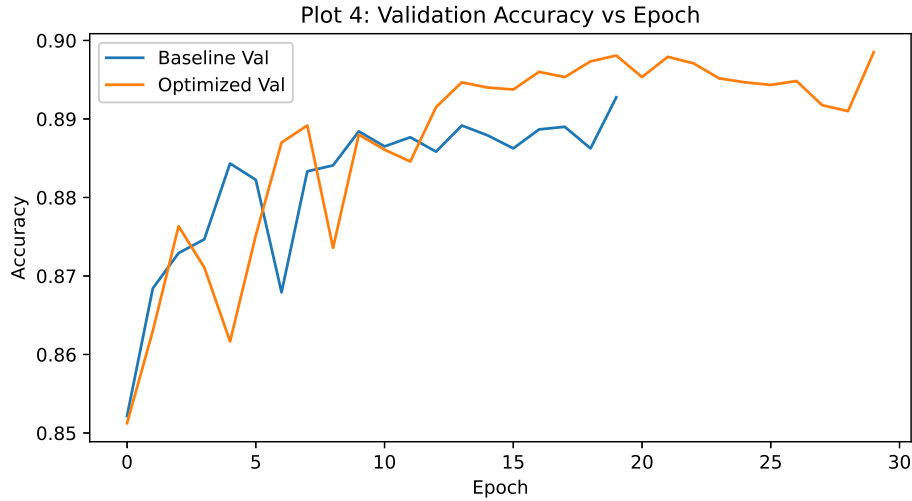


Figure 4: Validation Accuracy vs Epoch

**Inference:** The validation accuracy of baseline keeps fluctuating around 88-89%, but drops right after. The validation accuracy of Optimised model starts slightly lower than that of Baseline model, but stabilises and reaches higher near around 89.8% around 18-20 epochs(approx), due to better hyperparameter tuning. The generalisation gap in Optimised model is lesser than that of Baseline model.

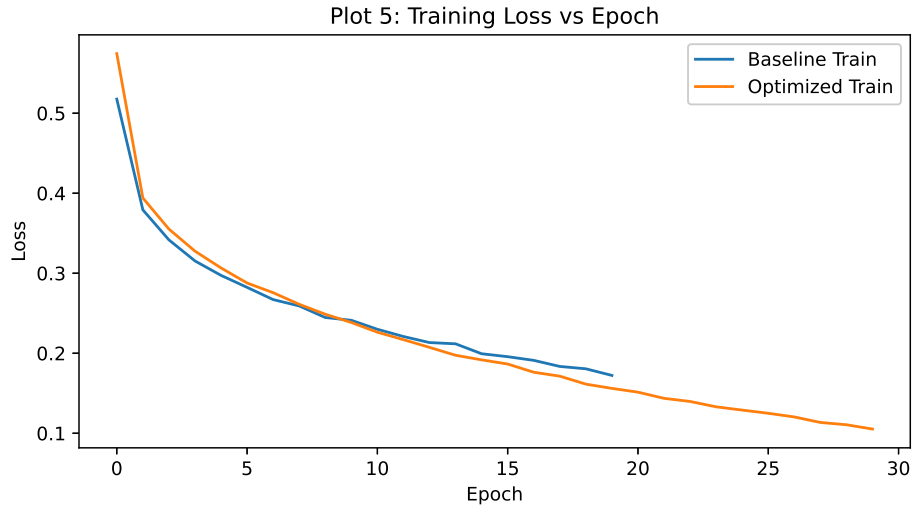


Figure 5: Training Loss vs Epoch

**Inference:** Training loss decreases for both models as training goes on. The final training loss is around 0.103 for optimised model (around epoch 30) and around 0.2 for baseline model (at epoch 20). The optimised model achieves a lower training loss due to higher model capacity and longer training duration.

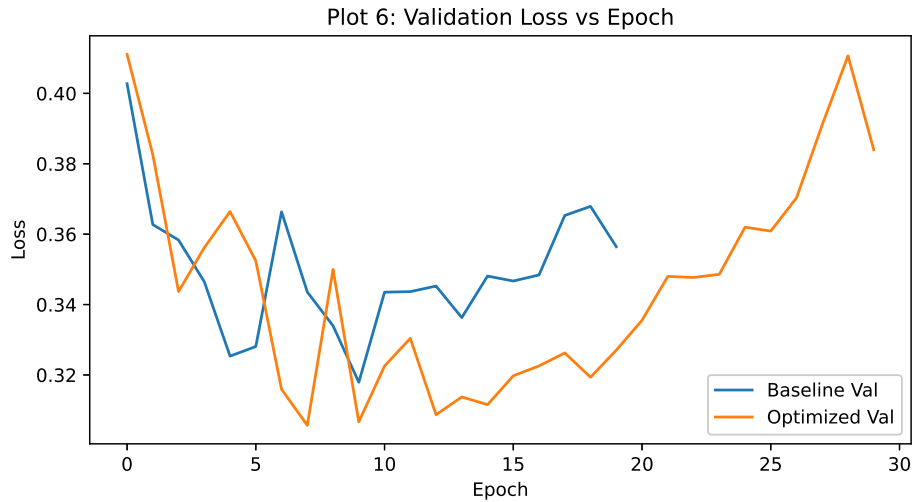


Figure 6: Validation Loss vs Epoch

**Inference:** Validation loss for baseline model reaches a minimum of (approx) 0.32 after 10th epoch (say around 11th epoch), but starts increasing after due to overfitting. Optimised model achieves a lower validation loss of (approx) 0.305 around the 6th epoch and maintains a lower loss line for most epochs. We can infer that hyperparameter tuning successfully reduced generalisation error on unseen data.

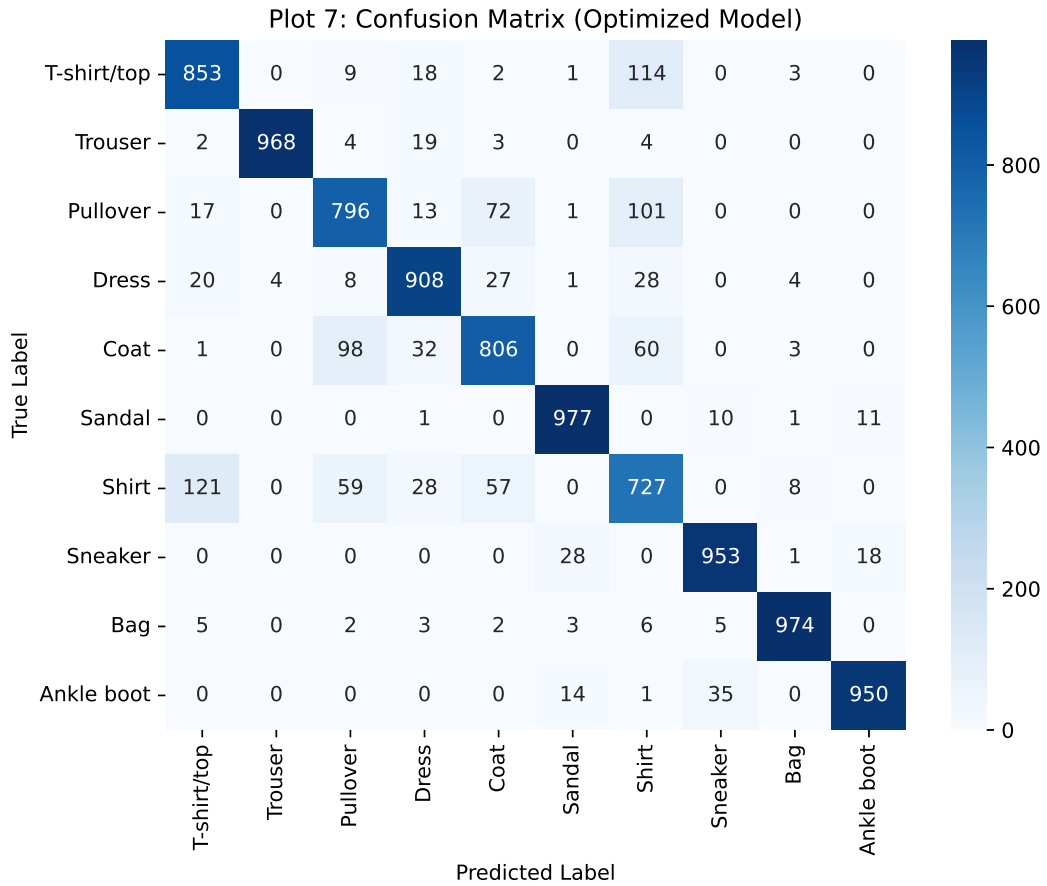


Figure 7: Confusion Matrix (Optimized Model)

**Inference:** The confusion matrix shows excellent performance in classifying easily distinguishable classes such as Trousers (968 true), Bags (974 true), etc... The major errors made by the model are with regards to upper body clothing, for instance, Shirts are confused for T-shirt/top (121), Pullovers (59) and Coats (57)- False positives. (due to visual similarity between the classes and model's lack of spatial feature extraction.)

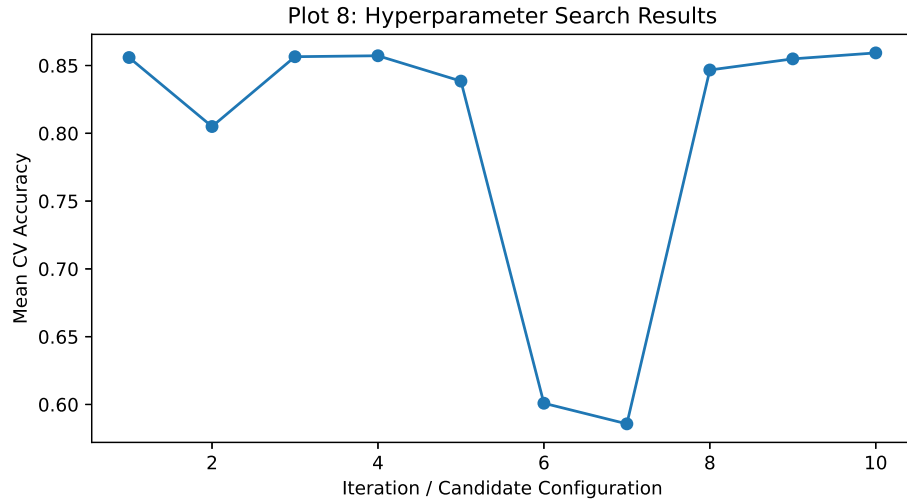


Figure 8: Hyperparameter Search Results

**Inference:** Cross validation accuracy varies across the 10 iterations. During iterations 1,2,3,4,8,9,10 the CV Accuracy varies in the range 0.80 to 0.86 (80-86%). We can see that there is a sudden drop during iterations 6-7. This implies that choosing the right Hyperparameter is important to prevent training failure.

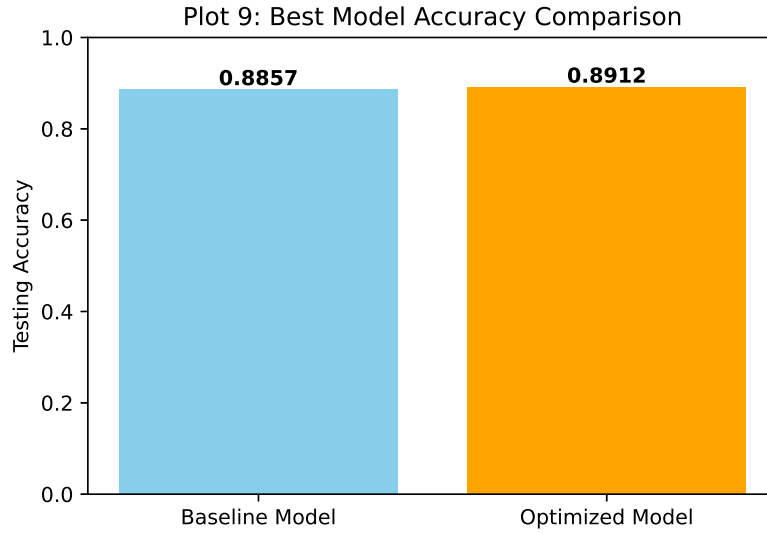


Figure 9: Baseline vs. Optimized Model Testing Accuracy

**Inference:** The final testing accuracies for baseline model and optimised model are 88.57% for baseline model and 89.12% for optimised model. The optimised model achieves higher accuracy than baseline, while also maintaining a lower validation loss and better resistance to overfitting(lesser generalisation gap) than baseline.

## 9. Results

### Best Hyperparameters



|                           |         |
|---------------------------|---------|
| Hidden Layers             | 2       |
| Hidden Neurons            | 256     |
| Learning Rate             | 0.001   |
| Batch Size                | 32      |
| Optimizer                 | Adam    |
| Activation Function       | sigmoid |
| Epochs                    | 30      |
| Dropout                   | 0.0     |
| Cross-validation Accuracy | 85.93%  |
| Testing Accuracy          | 89.12%  |

### Performance Comparison

| Metric        | Baseline | Optimized |
|---------------|----------|-----------|
| Accuracy      | 88.57%   | 89.12%    |
| Precision     | 88.91%   | 89.15%    |
| Recall        | 88.57%   | 89.12%    |
| F1-score      | 88.65%   | 89.10%    |
| Training Time | 108s     | 153s      |

## 10. Discussion

Answer the following:

### 1. Which optimization method was used?

**Answer:** RandomSearchCV and Adam were used.

- **RandomSearchCV:** Used for hyperparameter tuning with 5-fold cross validation. Instead of checking thousands of combinations, it randomly sampled 10 combinations to find the best setup efficiently.
- **Adam Optimizer:** Used for model training with a learning rate of 0.001 to perform back-propagation and update the weight parameters.

### 2. Which hyperparameters were selected?

**Answer:**

- Hidden Layers: 2
- Neurons per Layer: 256
- Activation Function: Sigmoid
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 32
- Epochs: 30
- Dropout Rate: 0.0

### 3. Did optimization improve performance?

**Answer:** Yes, hyperparameter optimization improved performance. The overall testing accuracy increased from 88.57% in the baseline model to 89.12% in the optimized model. Also, the optimised model achieved a lower validation loss (approx 0.305) as compared to that of baseline model (approx 0.318) and demonstrated better stability against overfitting.

4. **Which hyperparameter had the greatest impact?**

**Answer:** The Learning Rate had the greatest impact on model performance. In plot 8, setting a higher learning rate of 0.1 caused a drop/optimisation failure (during the 6-7 iterations) with mean CV accuracy dropping to approximately say, 58%.

5. **Compare Grid Search and Randomized Search.**

**Answer:**

- Grid Search: Exhaustively evaluates every single parameter combination within a predefined grid. It guarantees finding the absolute best combination, but it is computationally very expensive and slow for large search spaces.
- Randomized Search: Randomly samples a fixed number of parameter combinations (e.g. here, 10 combinations  $\times$  5 folds = 50 total fits in this setup). It reduces computing time. Also explores a broad parameter space, finding near-optimal setups.

6. **Recommend the best MLP configuration.**

**Answer:**

## 11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

## 12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.